Actividad III — Del incidente a la recuperación (Modelo B)

Materia: Criptografía

NRC: 1989

Profesor: Ricardo Luis Villanueva

Integrantes: Joshua Lobo Rios y Santiago Vengoechea Noriega

Fecha: noviembre de 2026

1. Introducción

Este trabajo simula, en un laboratorio aislado, el ciclo completo de un incidente de ransomware: la infección, el intercambio de una semilla maestra, el cifrado de los archivos de la víctima y las dos formas de recuperarlos. Sobre esa base construimos un intermediario (MITM) que roba la semilla y, por último, una defensa con firmas digitales que impide ese robo.

Elegimos el Modelo B, en el que la víctima genera la semilla maestra sV y se la entrega protegida al servidor. En el informe explicamos cómo quedó construida la aplicación, cómo se relaciona cada componente con los pasos del protocolo y qué observamos en las cuatro pruebas pedidas.

Servidor, intermediario y víctima corrieron en contenedores Docker separados, conectados por una red interna (`laboratorio`). La investigación y el descifrado del intermediario (M6) se ejecutaron en el anfitrión, con el servidor apagado y usando solo copias de los archivos cifrados. En las cuatro vías de recuperación, los archivos obtenidos resultaron idénticos byte a byte a los originales.

El reparto del trabajo fue el siguiente: Joshua se encargó del servidor y del núcleo criptográfico; Santiago, del cliente de la víctima, la capa de transporte y la investigación. El intermediario y las firmas digitales los diseñamos y programamos entre los dos, porque tocan ambos extremos del protocolo.

2. El modelo elegido y los parámetros

En el Modelo B la víctima genera la semilla sV con la función del profesor `generar_semilla_maestra()`, la cifra con OAEP.Enc(pkS, sV) y la envía al servidor. El servidor la abre con su clave privada skS y la guarda asociada al identificador de la víctima (idV), junto con la clave pública pkV. La recuperación por pago es igual en ambos modelos: el servidor devuelve la semilla cifrada con la clave registrada de la víctima, OAEP.Enc(pkV, sV).

Usamos los mismos parámetros criptográficos en todo el proyecto, como pide el enunciado:

- RSA de 2048 bits con OAEP y SHA-256 (MGF1 con SHA-256) para proteger la semilla maestra en el transporte.
- AES-256-GCM para cifrar cada archivo, con un nonce aleatorio de 12 bytes y una etiqueta de autenticación de 16 bytes.
- Una clave distinta por archivo, derivada como Ki = HMAC-SHA256(sV, i), donde i es un contador escrito en 8 bytes big-endian. El resultado de 32 bytes se usa directamente como clave AES-256. No se pide al servidor ninguna clave por archivo.

- Datos asociados (AAD) que atan cada cifrado a su dueño y a su posición: idV en UTF-8 seguido del contador i en 8 bytes big-endian. Como i siempre ocupa los últimos 8 bytes, la codificación no es ambigua.
- RSA-PSS con SHA-256 para las firmas de la defensa, con pares de firma independientes de los de cifrado.

Derivar las claves con HMAC permite guardar una sola semilla y regenerar con ella las claves de todos los archivos. La etiqueta de AES-GCM cumple un doble papel: además de descifrar, comprueba que el contenido no fue alterado y que la clave usada es la correcta. Como cada archivo tiene su propia clave y su nonce es aleatorio, nunca se repite un nonce con la misma clave.

3. Componentes de la aplicación

La aplicación se organiza en módulos con responsabilidades separadas:

Componente	Archivo	Qué hace
Servidor	servidor/servidor.py	Autentica y registra a cada víctima, recibe y guarda sV, atiende la recuperación por pago y firma su clave pública
Víctima	victima/victima.py	Genera sus pares RSA, se registra con pkV firmada, verifica pkS, genera y entrega sV, cifra y recupera por pago
Intermediario	mitm/mitm.py	Proxy que sustituye claves, intercepta la semilla y descifra copias con ella
Investigación	investigacion/investigar.py	Recupera sV por fuerza bruta, sin el servidor
Núcleo criptográfico	comun/crypto_common.py	RSA-OAEP, derivación Ki con HMAC-SHA256 y AES-256-GCM con AAD
Firmas	comun/firmas.py	Firma y verificación RSA-PSS de claves públicas
Transporte	comun/transporte.py	Mensajes JSON sobre TCP con un prefijo de longitud de 4 bytes
Archivos	comun/archivos.py	Impide que un nombre de archivo manipulado escriba fuera de la carpeta de salida
Generador	comun/generador_semillas.py	Suministrado por el profesor, sin modificar

Mensajes. Todos los mensajes son objetos JSON con un campo tipo y llevan el idV:

Mensaje	Emisor	Campos

Mensaje	Emisor	Campos
registro	Víctima	idV, pkv_n, pkv_e, pkv_firma
registro_ok	Servidor	pks_n, pks_e, pks_firma
entrega_semilla	Víctima	idV, encapsulacion_b64 = OAEP.Enc(pkS, sV)
entrega_ok	Servidor	idV
recuperar_pago	Víctima	idV (el evento de pago)
recuperacion	Servidor	idV, encapsulacion_b64 = OAEP.Enc(pkV, sV)
error	Cualquiera	detalle

Qué se guarda en disco. El servidor conserva su par RSA de cifrado, su par RSA de firma, las claves de verificación de las víctimas (confianza_victimas/<idV>.pem) y un registro idV → {pkV, sV, pagado} en JSON. Todo vive en un volumen de Docker que sobrevive al reinicio. El registro se escribe de forma atómica (archivo temporal y reemplazo), y una segunda entrega para el mismo idV se rechaza, así que repetir el registro no crea otra maestra.

La víctima guarda, por idV, su par RSA de cifrado y su par de firma, además de la clave de verificación del servidor. Nunca escribe en disco la semilla sV, las claves Ki ni la encapsulación OAEP original. Cada archivo cifrado se guarda como un JSON autocontenido (cifrado_<i>.json) con idV, el contador i, el nombre original, el nonce, la etiqueta y el texto cifrado, todo en base64. Con esos datos y la semilla, cualquier archivo se puede reconstruir.

4. Relación con los pasos del protocolo

La tabla conecta cada paso de los diagramas del enunciado con el lugar del código donde ocurre:

Paso	Dónde ocurre en el código
1. Iniciar y registrar	victima.comando_preparar crea los pares; comando_cifrar llama a registrar; el servidor responde en manejar_registro, tras autenticar_clave_victima
2. Establecer la maestra (B)	La víctima verifica pkS en obtener_clave_servidor, genera sV y la cifra con encapsular_oaep; el servidor la abre en manejar_entrega_semilla
3. Cifrar y guardar	La víctima deriva Ki con derivar_clave_Ki, cifra con cifrar_con_aad y guarda con escribir_cifrado
4. Cerrar y reiniciar	El servidor recarga sus pares RSA, las claves de confianza y el registro desde el volumen; la víctima conserva solo sus pares RSA y los cifrados
5a. Recuperación por pago	victima.comando_recuperar_pago y servidor.manejar_recuperacion_pago, que usa siempre la pkV registrada

Paso	Dónde ocurre en el código
5b. Recuperación por investigación	investigacion/investigar.py
6. Descifrar y verificar	La víctima, la investigación y el intermediario regeneran Ki y usan descifrar_con_aad, que verifica la etiqueta GCM; un archivo con etiqueta inválida se rechaza y se informa

Qué adaptamos del material de referencia. Partimos del Cuaderno Colab 1, en particular de las secciones de RSA-OAEP (sección 7) y de la aplicación cliente-servidor (sección 8). De allí adaptamos las operaciones de encapsulación OAEP y la comunicación por sockets con un prefijo de longitud. Hicimos tres cambios importantes:

1. Reemplazamos secrets.token_bytes(32) por generar_semilla_maestra(), que es la función que debíamos usar.
2. Añadimos datos asociados (AAD) a AES-GCM, llamando a aes.update(aad) antes de cifrar o verificar, para atar cada archivo a su idV y a su contador.
3. Cambiamos los mensajes a JSON con un campo tipo y añadimos persistencia en disco y las firmas RSA-PSS, que no estaban en el cuaderno.

5. Prueba 1 — Cifrado y pago

Objetivo. Mostrar que los archivos cifrados permanecen en disco tras cerrar y reiniciar los procesos, y que el pago permite recuperar la semilla y los originales (pasos 1–4, 5a y 6).

Procedimiento. Levantamos el servidor en el proyecto de Docker p1. Copiamos la clave de verificación del servidor a la víctima y la de la víctima v-p1 al servidor. La víctima se registró, el servidor verificó la firma de pkV, la víctima verificó la firma de pkS, generó la semilla, la entregó cifrada y cifró los dos archivos de prueba: uno de texto y uno binario de 1 MiB. Después cerramos y reiniciamos el servidor y pedimos la recuperación por pago.

Resultado. Tras el reinicio, el servidor mostró Clave RSA del servidor cargada. y Clave RSA de firma cargada., lo que confirma que recargó sus claves desde el volumen en lugar de generar unas nuevas. También conservó el registro de v-p1 con su semilla. La recuperación por pago devolvió sV y con ella se descifraron ambos archivos.

Traza del servidor:

```
Clave RSA del servidor generada y guardada.
Clave RSA de firma generada y guardada.
Servidor escuchando en 0.0.0.0:5000
Conexión aceptada de: ('172.20.0.4', 48286)
Víctima registrada (firma de pkV verificada): v-p1
Semilla maestra recibida y guardada para: v-p1
Clave RSA del servidor cargada.
Clave RSA de firma cargada.
Servidor escuchando en 0.0.0.0:5000
Conexión aceptada de: ('172.20.0.3', 55574)
Recuperación por pago simulado para: v-p1
```

Comparación con los originales (SHA-256):

```
binario.bin         IDENTICO
documento.txt       IDENTICO
```

La evidencia completa está en `evidencias/p1_pago/`: traza del servidor, cifrados, recuperados y archivo de comparación.

6. Prueba 2 — Investigación

Objetivo. Con el servidor apagado y usando solo la información permitida, recuperar la misma semilla y comparar el costo estimado con el medido (paso 5b).

6.1 Por qué el ataque es posible

La función que genera la semilla hace, en esencia, lo siguiente:

```
configuracion = secrets.randbelow(9) + 16   # un entero entre 16 y 24
valor = secrets.randbelow(1 << configuracion)
return valor.to_bytes(32, "big")
```

Aunque la semilla ocupa 32 bytes, el valor que contiene siempre es menor que 2^configuracion, y `configuracion` nunca pasa de 24. Por lo tanto, el valor siempre es menor que 2^24. `configuracion` no agranda el conjunto de semillas posibles, porque [0, 2^16) está contenido en [0, 2^24); solo cambia la probabilidad de cada valor. El conjunto completo es {0, 1, …, 2^24 − 1}: 16 777 216 valores, es decir, 24 bits de entropía como máximo, muy lejos de los 256 bits de una clave AES-256.

Además, la distribución no es uniforme. Un valor menor que 2^16 puede salir con cualquiera de las nueve configuraciones, mientras que uno mayor que 2^23 solo puede salir con una. Los valores pequeños son mucho más probables, y eso hace el ataque todavía más barato.

6.2 Cómo recuperamos la semilla

Recorremos los candidatos de 0 a 2^24 − 1 en orden creciente. Para cada candidato reconstruimos la semilla de 32 bytes, derivamos la clave del archivo de referencia e intentamos verificar su etiqueta GCM con su nonce y sus datos asociados. Esa verificación es el criterio para reconocer la semilla correcta: una etiqueta válida coincide por azar con probabilidad 2^−128, así que en cuanto una verificación pasa damos la semilla por correcta. No hace falta ningún otro indicio.

Recorrer en orden creciente es la estrategia óptima para esta distribución, porque prueba primero los valores más probables. Añadimos dos mejoras más:

- Usamos como referencia el archivo con el texto cifrado más corto, porque verificar la etiqueta cuesta en proporción a su tamaño. Cualquier archivo sirve, ya que su clave y sus datos asociados dependen solo de su propio contador.
- Repartimos el rango entre varios procesos con `multiprocessing`.

Como medida del trabajo usamos la posición de la semilla en el recorrido: para encontrar la semilla s hay que probar al menos los s + 1 candidatos de 0 a s. En modo multiproceso otros procesos pueden haber probado algunos candidatos más antes de detenerse; esos no se cuentan, así que la cifra es una cota inferior.

Durante el ataque solo usamos información permitida: el código de la aplicación y del generador, y los archivos cifrados con sus metadatos públicos. Nunca usamos el registro del servidor, claves privadas, la semilla guardada ni los archivos originales, que se reservaron para la comparación final.

6.3 Mediciones

Métrica	Valor
Entorno	Windows 11, Python 3.12.10, 12 procesos
Espacio de búsqueda	2^24 = 16 777 216 candidatos
Archivo de referencia	41 bytes de texto cifrado (documento.txt, i = 1)
Semilla encontrada	16 718 360 (0xff1a18)
Candidatos hasta la semilla (s + 1)	16 718 361 (99,6 % del espacio)
Percentil de la semilla	99,96 %
Tiempo en recuperar sV	324,437 s (5,4 min)
Tasa	51 530 candidatos por segundo
Peor caso estimado a esa tasa	326 s (5,4 min)
Tiempo en descifrar los archivos	0,188 s

Los valores salen de `evidencias/p2_investigacion/mediciones.csv` y de salida.txt.

6.4 Estimación frente a lo observado

Estimación. El peor caso exige probar los 2^24 candidatos. El caso promedio no es la mitad de ese espacio, porque la distribución no es uniforme. Si c es uniforme entre 16 y 24 y v es uniforme en [0, 2^c), el valor esperado de la semilla es:

$$E[v] = \frac{1}{9}\sum_{c=16}^{24}\frac{2^c - 1}{2} \approx 1{,}86 \times 10^6 \approx 2^{20{,}8}$$

Como la búsqueda es creciente, el número esperado de candidatos es E[v] + 1, unos 1,86 millones, cuatro veces menos que los 2^23 que daría una distribución uniforme. La mediana es de unos 399 000 candidatos, y el 22 % de las semillas caen por debajo de 2^16 = 65 536.

Observado. Nuestra semilla fue 16 718 360, y hubo que probar 16 718 361 candidatos, el 99,6 % del espacio. Es un caso extremo: solo la configuración 24 puede producir valores mayores que 2^23, y la probabilidad de obtener una semilla igual o mayor que esta es de apenas 0,04 % (1 de cada 2 566 víctimas). Por eso la búsqueda tardó mucho más que el caso típico, pero confirma la cota superior: incluso cerca del peor caso, la semilla se recuperó en 324 s (5,4 min). A la tasa medida de 51 530 candidatos por segundo, la mediana tomaría unos 8 s, el caso esperado unos 36 s y el peor caso 326 s, que coincide con lo observado. Como esta búsqueda fue larga, el costo de arrancar los 12 procesos es despreciable y la tasa refleja bien la velocidad real.

Costo de recuperar sV frente al de descifrar. Recuperar la semilla tomó 324,437 s; descifrar los dos archivos con la semilla ya conocida tomó 0,188 s, unas 1 700 veces menos. La búsqueda se hace una sola

vez y con el archivo más pequeño, así que su costo no depende de cuántos archivos tenga la víctima. El descifrado sí crece de forma lineal con el número y el tamaño de los archivos. Por eso, cuantos más archivos haya, más se amortiza el costo de la fuerza bruta.

La comparación final confirmó que los archivos recuperados por investigación son idénticos a los originales:

```
binario.bin         IDENTICO
documento.txt       IDENTICO
```

7. Prueba 3 — Intermediario (MITM)

Objetivo. Que un tercero intercepte la semilla durante su entrega inicial, deje que la víctima termine su cifrado sin darse cuenta y luego recupere los archivos sin pagar.

Procedimiento. En el proyecto `p3`, la víctima `v-p3` se conectó al intermediario (servicio `mitm`, puerto 5001), y este al servidor, de modo que todos los mensajes pasaron por él. El intermediario genera su par RSA (pkM, skM) al arrancar. En esta prueba la víctima acepta la clave recibida sin verificar firmas (`--sin-firmas`), como pide M2. El ataque, paso a paso:

- **M2 (sustitución).** El intermediario reenvió al servidor el registro de la víctima, con idV y pkV, sin cambios. A la víctima le entregó su propia clave pkM en lugar de pkS, y guardó la pkS verdadera. **La clave sustituida es pkS.**
- **M3 (intercepción).** La víctima cifró la semilla con pkM creyendo que era del servidor. El intermediario la abrió con skM, guardó la semilla en `mitm/datos/v-p3.json` y la volvió a cifrar con la pkS real.
- **M4 (reenvío).** El servidor recibió la entrega como si viniera de la víctima, la abrió con skS, la guardó bajo `v-p3` y confirmó. La confirmación se reenvió a la víctima, que no notó nada.
- **M5 (cifrado).** La víctima derivó sus claves y guardó los archivos cifrados. Cerramos víctima y servidor; los cifrados permanecieron en disco.
- **M6 (recuperación sin pago).** Con el servidor apagado, entregamos al intermediario copias de los cifrados. Con la semilla interceptada regeneró las claves, descifró, verificó GCM y obtuvo los originales.

La traza del intermediario muestra la sustitución, la apertura y el reenvío, y la del servidor muestra la confirmación final:

```
Proxy de laboratorio escuchando en 0.0.0.0:5001
Servidor de destino: servidor:5000
Conexión de laboratorio desde ('172.21.0.4', 43940)
M2: clave pública sustituida (pkS -> pkM) para v-p3
M3: maestra interceptada de v-p3
   sV = 0000000000000000000000000000000000000000000000000000000000000e7015
   guardada en /app/mitm/datos/v-p3.json
M3: maestra re-cifrada con la pkS real y reenviada.

----

Clave RSA del servidor generada y guardada.
Clave RSA de firma generada y guardada.
Servidor escuchando en 0.0.0.0:5000
Conexión aceptada de: ('172.21.0.3', 44744)
Víctima registrada (firma de pkV verificada): v-p3
```

```
Semilla maestra recibida y guardada para: v-p3
```

En esta prueba el servidor sí verificó la firma de pkV y la aceptó. En el Modelo B el intermediario no necesita tocar pkV: le basta con sustituir pkS ante la víctima. Por eso la defensa que detiene este ataque es la verificación de pkS en la víctima, que aquí estaba desactivada.

Por qué RSA-OAEP no evitó el robo. OAEP garantiza que solo quien tiene la clave privada correspondiente puede abrir el mensaje cifrado. No dice nada sobre a quién pertenece la clave pública usada. La víctima cifró con pkM sin comprobar de quién era, así que el intermediario, dueño legítimo de skM, pudo abrir la entrega sin dificultad. OAEP protege la confidencialidad del secreto en tránsito, pero no la autenticidad de la clave pública, y ese es el hueco que explota el MITM.

Los archivos recuperados por el intermediario resultaron idénticos a los originales:

```
binario.bin         IDENTICO
documento.txt       IDENTICO
```

8. Prueba 4 — Firmas digitales

Objetivo. Impedir la sustitución de claves de la prueba anterior, de modo que cada participante detecte el engaño antes de exponer la semilla o de registrar una clave falsa.

8.1 Diseño de la defensa

- **Esquema de firma.** Usamos RSA-PSS con SHA-256 (biblioteca PyCryptodome, `Crypto.Signature.pss`). Servidor y víctima tienen cada uno un par de firma de 2048 bits, independiente de su par de cifrado: el par de cifrado solo protege la semilla con OAEP, y el par de firma solo autentica claves. OAEP se mantiene para proteger la maestra; la firma no lo sustituye.
- **Quién firma y quién verifica.** El servidor firma su clave de cifrado pkS y la víctima verifica esa firma. La víctima firma su clave de cifrado pkV y el servidor verifica esa firma. La autenticación funciona en los dos sentidos.
- **Qué se firma.** Una serialización JSON canónica (claves ordenadas, sin espacios, UTF-8) de cinco campos: el protocolo (`actividad-iii-v1`), el rol (`servidor` o `victima`), el idV y los parámetros n y e de la clave de cifrado. Así la firma ata la clave recibida a la identidad de quien la emite y al idV del incidente. Una firma hecha para una víctima no sirve para otra, y una firma del rol `victima` no sirve como si fuera del servidor.
- **De dónde sale la confianza.** Las claves de verificación se intercambian antes del protocolo, por un canal aparte que el intermediario no controla. Copiamos `clave_firma_publica.pem` del servidor a `victima/datos/`, y la clave de verificación de cada víctima al servidor como `confianza_victimas/<idV>.pem`. Cada lado verifica siempre con la clave que ya tenía e ignora cualquier clave de verificación que llegue dentro del intercambio. Estas claves están en disco, así que la defensa sigue funcionando después del reinicio.
- **Cuándo se verifica.** El servidor verifica la firma de pkV en `autenticar_clave_victima`, antes de crear el registro. Si la firma falta o es inválida, o si no hay clave de confianza para ese idV, responde `error` y no registra a la víctima. La víctima verifica la firma de pkS en `obtener_clave_servidor`, antes de usar pkS para cifrar la semilla. Si falla, detiene el intercambio y no envía `entrega_semilla`. Nunca continúa con una simple advertencia.

Como el servidor solo registra una pkV autenticada, la recuperación por pago siempre cifra la semilla con la clave legítima de la víctima.

8.2 Resultados

En el proyecto `p4`, con una víctima nueva para cada caso y el mismo intermediario, probamos cuatro casos:

Caso	idV	Vía	Comprobación que falla	Mensaje que ya no se envía	Resultado
1. Sustitución de pkS	v-p4-mitm	Intermediario	Víctima, `obtener_clave_servidor`	`entrega_semilla`	Rechazado
2. Firma propia del intermediario	v-p4-firma-mitm	Intermediario (`MITM_FIRMAR_PKM=1`)	Víctima, `obtener_clave_servidor`	`entrega_semilla`	Rechazado
3. Sustitución de pkV	v-p4-pkv	Intermediario (`MITM_SUSTITUIR_PKV=1`)	Servidor, `autenticar_clave_victima`	`registro_ok`	Rechazado
4. Intercambio legítimo	v-p4-legitima	Servidor directo	Ninguna	Ninguno	Cifrado, reinicio y pago completos

Trazas de la víctima en los casos 1 y 2:

```
Container p4-victima-run-81b9aaf73cbe Creating
 Container p4-victima-run-81b9aaf73cbe Created
RECHAZO: verificar_firma_clave(clave de confianza, 'servidor', 'v-p4-mitm', pkS recibida, pks_firma) devolvió False. No se envía 'entrega_semilla'.
Error: Firma del servidor inválida: se cancela el intercambio y NO se envía la semilla maestra.

-----

Container p4-victima-run-34ed0e4f0d8b Creating
 Container p4-victima-run-34ed0e4f0d8b Created
AVISO: la respuesta trae su propia clave de verificación; se ignora y se usa solo la clave de confianza local.
RECHAZO: verificar_firma_clave(clave de confianza, 'servidor', 'v-p4-firma-mitm', pkS recibida, pks_firma) devolvió False. No se envía 'entrega_semilla'.
Error: Firma del servidor inválida: se cancela el intercambio y NO se envía la semilla maestra.
```

Traza del servidor, que confirma el desenlace de los cuatro casos:

```
Clave RSA del servidor generada y guardada.
Clave RSA de firma generada y guardada.
Servidor escuchando en 0.0.0.0:5000
Conexión aceptada de: ('172.22.0.3', 52378)
Víctima registrada (firma de pkV verificada): v-p4-mitm
```

```
Conexión aceptada de: ('172.22.0.3', 46522)
Víctima registrada (firma de pkV verificada): v-p4-firma-mitm
Conexión aceptada de: ('172.22.0.3', 34294)
Registro rechazado: Firma de pkV ausente o inválida: se rechaza el registro.
Conexión aceptada de: ('172.22.0.4', 50976)
Víctima registrada (firma de pkV verificada): v-p4-legitima
Semilla maestra recibida y guardada para: v-p4-legitima
Clave RSA del servidor cargada.
Clave RSA de firma cargada.
Servidor escuchando en 0.0.0.0:5000
Conexión aceptada de: ('172.22.0.3', 41830)
Recuperación por pago simulado para: v-p4-legitima
```

`v-p4-mitm` y `v-p4-firma-mitm` quedaron registradas sin semilla, porque la víctima se detuvo antes de enviarla. `v-p4-pkv` ni siquiera se registró. `v-p4-legitima` completó el ciclo, incluido el reinicio y la recuperación por pago, con archivos idénticos a los originales.

8.3 Por qué se rechaza cada ataque

En el **caso 1**, el intermediario cambia los parámetros de la clave por los de pkM, pero deja la firma original del servidor, hecha sobre la pkS verdadera. Al verificar esa firma con la clave de confianza, los datos firmados no coinciden con la clave recibida y la verificación falla.

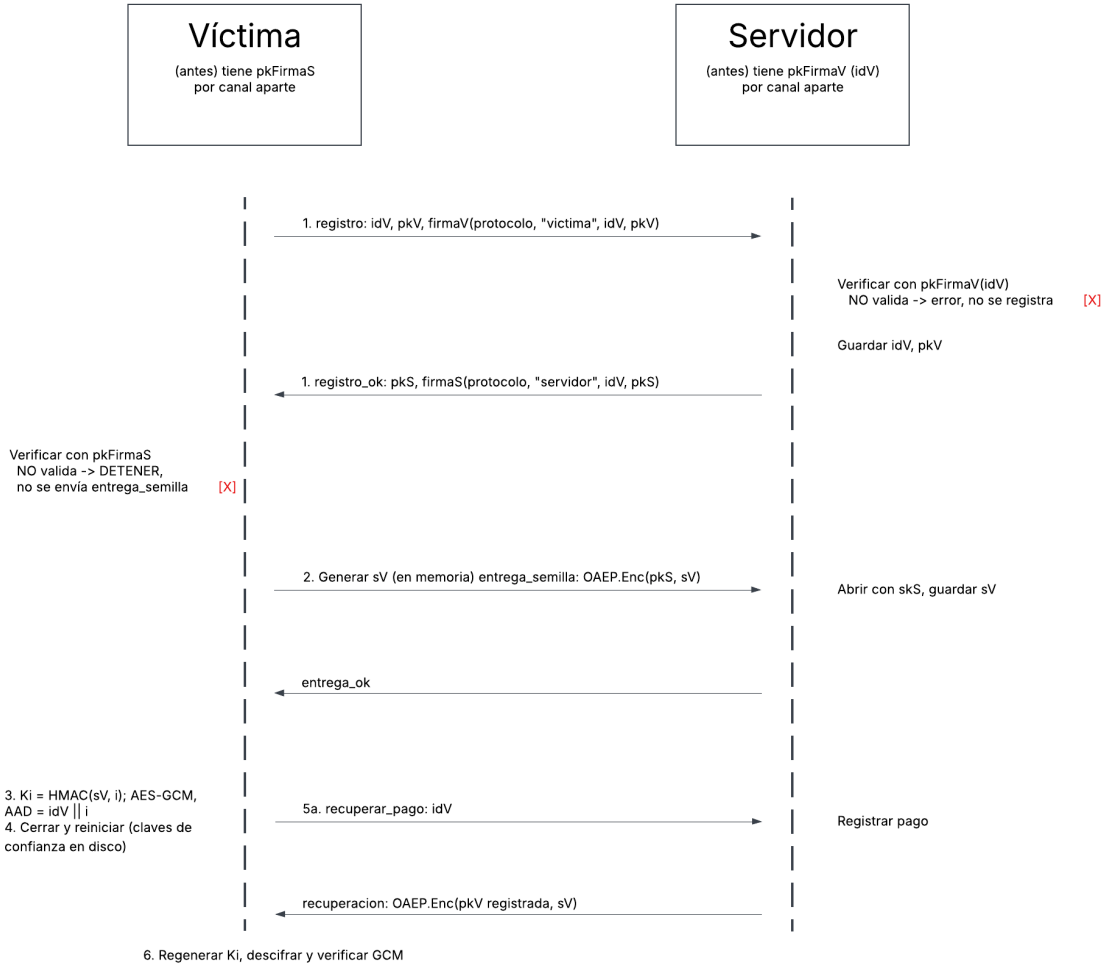
En el **caso 2**, el intermediario firma pkM con su propia clave de firma y adjunta su clave de verificación. Aun así la víctima lo rechaza, porque no verifica con la clave que entrega el atacante, sino con la del servidor real que ya tenía de antemano.

En el **caso 3**, el intermediario hace lo simétrico ante el servidor: cambia pkV por pkM y conserva la firma de la víctima. Esa firma se hizo sobre la pkV real, así que no verifica para pkM y el servidor rechaza el registro. En el Modelo B este ataque no le daría la semilla en la entrega inicial, pero si el servidor aceptara pkM, la recuperación por pago le entregaría la semilla cifrada al atacante.

De aquí sale la conclusión más importante de la prueba: **confiar solo en la firma recibida no basta.** Si la víctima aceptara la clave de verificación que llega en el mismo intercambio, el intermediario podría reemplazar a la vez la clave de cifrado y la que verifica la firma, como intenta en el caso 2, y pasar la comprobación. La seguridad no viene de la firma en sí, sino de anclar la confianza en claves de verificación legítimas, conocidas de antemano por un canal independiente. Frente a RSA-OAEP, las firmas añaden lo que OAEP no daba: la autenticación del origen de cada clave pública, que es justo lo que cerró el hueco aprovechado en la prueba 3.

8.4 Diagrama adaptado del Modelo B con la defensa

El ciclo de los pasos 1 a 6 se mantiene. La defensa añade una firma en `registro` y otra en `registro_ok`, y un punto de rechazo en cada lado antes de usar la clave recibida.



9. Comparación final de archivos recuperados

En las cuatro vías de recuperación, los archivos obtenidos coinciden exactamente con los originales reservados, verificados con SHA-256:

Prueba	binario.bin (1 MiB)	documento.txt
1. Pago	[IDÉNTICO]	[IDÉNTICO]
2. Investigación	[IDÉNTICO]	[IDÉNTICO]
3. MITM (M6)	[IDÉNTICO]	[IDÉNTICO]
4. Firmas, intercambio legítimo (pago)	[IDÉNTICO]	[IDÉNTICO]

Los archivos comparacion.txt de cada carpeta de evidencias/ respaldan esta tabla.

10. Conclusiones

El ciclo completo del Modelo B funciona de principio a fin y sobrevive al reinicio: el servidor recupera su estado desde el disco y la víctima recupera sus archivos sin haber guardado la semilla. El diseño no depende de información que viva solo en memoria.

La investigación dejó una lección de criptografía práctica: un algoritmo fuerte como AES-256 no sirve de nada si la clave proviene de una fuente con poca entropía. Al limitar las semillas a 24 bits, el generador convirtió una clave teóricamente irrompible en algo que se adivina en segundos en el caso típico y en pocos minutos en el peor. El análisis también mostró que el parámetro `configuracion` no aporta seguridad: no amplía el conjunto de valores posibles y, al concentrar la probabilidad en los valores pequeños, abarata todavía más la búsqueda.

El intermediario demostró que la confidencialidad no implica autenticidad: RSA-OAEP protege el secreto en tránsito, pero no impide que alguien se haga pasar por el servidor sustituyendo su clave pública. Las firmas digitales cerraron ese hueco en los dos sentidos: la víctima autentica la clave del servidor y el servidor la de la víctima. Pero solo funcionan porque cada lado ancla su confianza en una clave de verificación obtenida de antemano por un canal aparte. Si esa confianza se pudiera sustituir junto con todo lo demás, la defensa no serviría. El verdadero punto de seguridad no es firmar, sino tener una raíz de confianza que el atacante no pueda tocar.

11. Fuentes

- Cuaderno Colab 1 del curso: *RSA, fundamentos, ataques y cifrado híbrido*, secciones 7 (RSA-OAEP) y 8 (aplicación cliente-servidor).
- Documentación de PyCryptodome: módulos `Crypto.Cipher.PKCS1_OAEP`, `Crypto.Cipher.AES` (modo GCM), `Crypto.Hash.HMAC`, `Crypto.Hash.SHA256` y `Crypto.Signature.pss`.
- RFC 8017, PKCS #1 v2.2: RSA-OAEP y RSA-PSS.
- NIST SP 800-38D: modo de operación GCM.
- FIPS 198-1: HMAC.

Anexo: evidencias entregadas

La carpeta `evidencias/` del ZIP contiene, por prueba:

Carpeta	Contenido
pruebas_unitarias.txt	Salida de las 12 pruebas unitarias
p1_pago/	Originales, cifrados, recuperados, logs/servidor.log, comparacion.txt
p2_investigacion/	Cifrados copiados, recuperados, mediciones.csv, salida.txt, comparacion.txt
p3_mitm/	Cifrados, recuperados por el intermediario, logs/servidor.log, logs/mitm.log, comparacion.txt
p4_firmas/	Cifrados y recuperados del caso legítimo, salidas de la víctima por caso, trazas del intermediario por caso, logs/servidor.log, comparacion.txt